GEOLOGOS ECOSYSTEM: Complete API Reference

All 80+ Endpoints Documented

📚 TABLE OF CONTENTS

1. Research Engine (8001)
2. Wiki Platform (8002)
3. Comic Creator (8003)
4. Podcast Studio (8004)
5. Creative Suite (8005)
6. Story Engine (8006)
7. World Builder (8007)
8. Cosmic Archive (8008)
9. Blog Platform (3000)
10. Health & Status

🔍 RESEARCH ENGINE (Port 8001)

POST /api/v1/research/scrape

Scrape URL and add to research database

```
{
  "url": "https://example.com/article",
  "tags": ["ai", "research"]
}
```

Response: `{id, title, domain, content_hash}`

GET /api/v1/research/search

Search research sources

?query=machine%20learning&limit=20&include_mesh=true

Response: `{query, results: [], total}`

GET /api/v1/research/source/{source_id}

Get specific source Response: `{id, url, title, authors, published_date, content, ...}`

POST /api/v1/research/cite

Generate citation for source

```
{
  "source_id": "abc123",
  "format": "APA"  // APA|MLA|CHICAGO|HARVARD|IEEE|BIBTEX|CSL_JSON
}
```

Response: `{source_id, format, text, bibtex, in_text, created_at}`

GET /health

Response: `{status: "healthy", service: "research-engine"}`

📖 WIKI PLATFORM (Port 8002)

POST /api/v1/wiki/pages

Create wiki page

```
{
  "title": "Page Title",
```

```
  "content": "# Markdown content",

  "author": "Author Name",

  "tags": ["tag1", "tag2"],

  "references": ["other-page-slug"]

}
```

Response: `{id, slug, title, content, author, tags, ...}`

GET /api/v1/wiki/pages/{slug}

Get wiki page Response: `{id, slug, title, content, author, version, tags, ...}`

PUT /api/v1/wiki/pages/{slug}

Edit wiki page

```
{

  "content": "Updated content",

  "author": "Editor Name",

  "summary": "Fixed typos and added details"

}
```

Response: `{slug, version, updated_at, ...}`

GET /api/v1/wiki/search?q=query

Search wiki pages Response: `[{id, slug, title, excerpt}, ...]`

GET /api/v1/wiki/tags

Get all tags with counts Response: `{tag1: 5, tag2: 3, ...}`

GET /api/v1/wiki/pages/{slug}/backlinks

Get pages linking to this page Response: `["page-slug-1", "page-slug-2"]`

GET /api/v1/wiki/pages/{slug}/related

Get related pages Response: `[{id, slug, title}, ...]`

GET /api/v1/wiki/toc

Get table of contents Response: `{toc: "# TOC\n..."}`

GET /api/v1/wiki/export

Export entire wiki as JSON Response: `{pages: {...}, tags: {...}, exported_at}`

🎨 COMIC CREATOR (Port 8003)

POST /api/v1/comics/stories

Create new comic story

```
{
  "title": "Story Title",
  "description": "Story description",
  "genre": "sci-fi",
  "tags": ["tag1"]
}
```

Response: `{id, title, description, characters, pages, ...}`

POST /api/v1/comics/stories/{story_id}/characters

Add character to story

```
{
  "name": "Character Name",
  "description": "Character description",
```

```
  "visual_prompt": "Visual description for AI",

  "personality_traits": ["trait1", "trait2"]

}
```

Response: `{id, name, description, visual_prompt, ...}`

POST /api/v1/comics/stories/{story_id}/pages

Create comic page with AI panels

```
{

  "page_number": 1,

  "panel_descriptions": [

    "Panel 1 description",

    "Panel 2 description"

  ],

  "layout": "grid_2x2",  // single|two_columns|three_columns|grid_2x2|grid_3x3

  "theme": "standard"

}
```

Response: `{id, page_number, panels: [{id, image_url, dialogue, ...}], ...}`

GET /api/v1/comics/stories/{story_id}

Get story Response: `{id, title, description, characters, pages, ...}`

POST /api/v1/comics/stories/{story_id}/pages/{page_id}/citations

Add research citation to page

```
{
```

"citation": "Author (Year). Title. Source."

}

Response: `{status: "citation added"}`

GET /api/v1/comics/stories/{story_id}/render

Render all pages of story Response: `{story_id, pages_rendered, page_numbers}`

🎙️ PODCAST STUDIO (Port 8004)

POST /api/v1/podcasts/shows

Create podcast series

{

"title": "Show Title",

"description": "Show description",

"host": "Host Name",

"tags": ["tag1"]

}

Response: `{id, title, description, host, episodes, ...}`

POST /api/v1/podcasts/shows/{show_id}/episodes

Create episode (auto TTS + transcription)

{

"episode_number": 1,

"title": "Episode Title",

"script": "Full episode script text...",

```
  "narrator_voice": "default",  // Voice choice

  "tags": ["tag1"]

}
```

Response: `{id, episode_number, title, script, tracks, transcript, ...}`

GET /api/v1/podcasts/shows/{show_id}

Get podcast series Response: `{id, title, description, host, episodes, ...}`

POST /api/v1/podcasts/episodes/{episode_id}/render

Render/mix episode audio Response: `{status: "rendered", file: "episodes/xyz.wav"}`

🎵 CREATIVE SUITE (Port 8005)

POST /api/v1/music/compositions

Create music composition

```
{

  "title": "Composition Title",

  "genre": "ambient",

  "bpm": 120

}
```

Response: `{id, title, genre, bpm, tracks, duration_ms, ...}`

POST /api/v1/music/compositions/{comp_id}/tracks

Add music track

```
{
```

```
  "instrument": "synth",  // piano|guitar|synth|strings|drums|bass

  "notes": [[60, 500], [62, 500]]  // [note_number, duration_ms]

}
```

Response: `{id, instrument, notes, volume, effects, ...}`

POST /api/v1/music/compositions/{comp_id}/render

Render composition to audio Response: `{status: "rendered", file: "music/xyz.wav"}`

POST /api/v1/art/collections

Create art collection

```
{

  "title": "Collection Title",

  "description": "Collection description",

  "tags": ["tag1"]

}
```

Response: `{id, title, description, artworks, tags, ...}`

POST /api/v1/art/generate

Generate artwork with AI

```
{

  "title": "Artwork Title",

  "description": "Visual description",

  "style": "cyberpunk",  // comic|oil_painting|cyberpunk|watercolor|concept_art

  "width": 512,

  "height": 512
```

```
}
```

Response: `{id, title, image_url, prompt, style, generation_params, ...}`

📖 STORY ENGINE (Port 8006)

POST /api/v1/stories

Create new story

```
{
  "title": "Story Title",
  "universe": "nomadz",
  "description": "Story description",
  "tags": ["tag1"]
}
```

Response: `{id, title, universe, description, branches, timeline, ...}`

POST /api/v1/stories/{story_id}/scenes

Add story scene/node

```
{
  "node_type": "scene",  // scene|choice|event|revelation|climax|resolution
  "title": "Scene Title",
  "description": "Scene description",
  "content": "Scene content/text",
  "characters": ["char-id-1"],
```

```
  "locations": ["loc-id-1"],

  "timeline_point": 0

}
```

Response: `{id, type, title, content, choices, consequences, ...}`

POST /api/v1/stories/{story_id}/choices

Add branching choice

```
{

  "from_node_id": "node-1",

  "to_node_id": "node-2",

  "choice_text": "What the player/reader sees",

  "consequence": "What changes after this choice"

}
```

Response: `{status: "choice added"}`

POST /api/v1/stories/{story_id}/branches

Create story branch

```
{

  "title": "Branch Title",

  "starting_node_id": "node-1",

  "description": "Branch description"

}
```

Response: `{id, title, starting_node, nodes, ...}`

GET /api/v1/stories/{story_id}

Get story Response: `{id, title, branches, nodes, timeline, ...}`

GET /api/v1/stories/{story_id}/check

Check continuity Response: `{issues: [], valid: true}`

GET /api/v1/stories/{story_id}/summary

Get story summary Response: `{summary: "# Story Title\n..."}`

POST /api/v1/stories/{story_id}/canonical/{branch_id}

Mark branch as canonical Response: `{status: "marked canonical"}`

🌍 WORLD BUILDER (Port 8007)

POST /api/v1/worlds

Create world

```
{
  "name": "World Name",
  "universe": "nomadz",
  "width": 100,
  "height": 100
}
```

Response: `{id, name, dimensions, locations, regions, biome_map, ...}`

POST /api/v1/worlds/{world_id}/locations

Add location to world

```
{
```

```
  "name": "Location Name",

  "location_type": "city",  //
city|village|dungeon|shrine|research_center|outpost|ruin|natural_landmark|dimensional_portal

  "x": 50.0,

  "y": 50.0,

  "z": 0.0,

  "biome": "forest",  // forest|desert|mountain|ocean|tundra|grassland|swamp|urban|void

  "description": "Location description"

}
```

Response: `{id, name, coordinates, biome, population, resources, ...}`

POST /api/v1/worlds/{world_id}/regions

Create region

```
{

  "name": "Region Name",

  "biome": "forest",

  "area_sq_km": 10000,

  "history": "Region history"

}
```

Response: `{id, name, biome, locations, area_sq_km, ...}`

POST /api/v1/worlds/{world_id}/connections

Connect locations (trade routes)

```
{
```

"location_a_id": "loc-1",

"location_b_id": "loc-2"

}

Response: `{status: "connected"}`

GET /api/v1/worlds/{world_id}

Get world Response: `{id, name, locations, regions, biome_map, ...}`

GET /api/v1/worlds/{world_id}/map

Get ASCII map visualization Response: `{map: "🌲🌲🏜️...\n..."}`

GET /api/v1/worlds/{world_id}/locations/{location_id}

Get location details Response: `{id, name, coordinates, biome, population, resources, connections, ...}`

🔑 COSMIC KEY ARCHIVE (Port 8008)

POST /api/v1/cosmic/archives

Create cosmic knowledge archive

{

"name": "Archive Name",

"encrypted": true

}

Response: `{id, name, truths, encrypted, mesh_peers, ...}`

POST /api/v1/cosmic/archives/{archive_id}/truths

Add cosmic truth

{

```
  "title": "Truth Title",

  "category": "consciousness",  //
creation|dimensions|consciousness|ethics|technology|spirituality|history|prophecy

  "content": "Truth content/description",

  "author": "Documented by",

  "access_level": "sacred"  // public|restricted|sacred|classified

}
```

Response: `{id, title, category, content, author, verified, ...}`

POST /api/v1/characters

Generate character

```
{

  "name": "Character Name",

  "archetype": "hero",  //
hero|mentor|trickster|shadow|innocent|lover|sage|magician|everyman|explorer

  "age": 30,

  "origin": "nomadz",

  "background": "Character background story",

  "appearance": "Character appearance description"

}
```

Response: `{id, name, archetype, traits, abilities, relationships, ...}`

GET /api/v1/characters/{char_id}/sheet

Get character sheet Response: `{sheet: "# Character Name\n..."}`

GET /api/v1/cosmic/archives/{archive_id}/export

Export archive Response: `{name, truths: {...}, categories: {...}, exported_at}`

📱 BLOG PLATFORM (Port 3000)

Next.js Blog - Standard REST endpoints

- GET / → Homepage with featured posts
- GET /blog → Blog archive
- GET /blog/{slug} → Individual post
- GET /api/posts → JSON list of posts
- GET /api/posts/{slug} → Post JSON
- POST /api/newsletter/subscribe → Newsletter signup

✅ HEALTH & STATUS

GET /health (All services)

All services respond to health checks:

localhost:8001/health

localhost:8002/health

localhost:8003/health

localhost:8004/health

localhost:8005/health

localhost:8006/health

localhost:8007/health

localhost:8008/health

Response: `{status: "healthy", service: "service-name"}`

🔄 COMMON PATTERNS

Error Response (All services)

```
{
  "detail": "Error message explaining what went wrong"
}
```

Pagination (Where applicable)

```
?limit=20&offset=0
```

Filtering

```
?tags=tag1,tag2&status=published
```

Sorting

```
?sort=created_at&direction=desc
```

🚀 QUICK EXAMPLE: FULL WORKFLOW

```
# 1. Research
curl -X POST localhost:8001/api/v1/research/scrape \
  -H "Content-Type: application/json" \
  -d '{"url": "https://example.com", "tags": ["research"]}'
# 2. Cite
curl -X POST localhost:8001/api/v1/research/cite \
  -H "Content-Type: application/json" \
```

```
  -d '{"source_id": "abc123", "format": "APA"}'

# 3. Create Story

curl -X POST localhost:8006/api/v1/stories \

  -H "Content-Type: application/json" \

  -d '{"title": "My Story", "universe": "nomadz", "description": "..."}'

# 4. Generate Character

curl -X POST localhost:8008/api/v1/characters \

  -H "Content-Type: application/json" \

  -d '{"name": "Hero", "archetype": "hero", ...}'

# 5. Create World

curl -X POST localhost:8007/api/v1/worlds \

  -H "Content-Type: application/json" \

  -d '{"name": "World", "universe": "nomadz", ...}'

# 6. Create Comic Story

curl -X POST localhost:8003/api/v1/comics/stories \

  -H "Content-Type: application/json" \

  -d '{"title": "Comic", "description": "..."}'

# 7. Generate Comic Pages (AI panels)

curl -X POST localhost:8003/api/v1/comics/stories/{story_id}/pages \

  -H "Content-Type: application/json" \

  -d '{"page_number": 1, "panel_descriptions": [...], "layout": "grid_2x2"}'

# 8. Add Citation
```

```
curl -X POST localhost:8003/api/v1/comics/stories/{story_id}/pages/{page_id}/citations \
  -H "Content-Type: application/json" \
  -d '{"citation": "Research citation"}'
# Everything now connected and published!
```

GEOLOGOS: Complete, documented, production-ready API. 80+ endpoints. One integrated vision.

🚀 **Build the future with this.**